浙江大学实验报告

专业：电子科学与技术
姓名：郑智瑄
学号：3230103913
日期：May 28, 2025
地点：东 4-216

课程名称：电子电路设计实验 II　指导老师：叶险峰/施红军/邓靖靖　实验类型：设计型
实验名称：实验设计报告　成　绩：　同组学生姓名：魏希浩

一、 实验目的

1. 学习掌握用 Arduino UNO 设计数字时钟;
2. 学习掌握 PCB 电路板的设计和制作;
3. 学习掌握 Arduino UNO 扩展板的设计与制作;
4. 学习掌握 DS1302 时钟芯片和 LCD1602 液晶显示屏的使用。

二、 实验任务与要求

1. 用 Arduino UNO 设计多功能数字时钟，要求实现功能：

- 显示时间、日期、星期和温度，并能断电保存时间
- 在整点、到达自定义闹钟时间、到达报警温度时报警
- 按键功能：按选择键进入设置时间功能；同时按 + - 键进入闹钟和报警温度设置功能；
- 再按选择键光标跳动，光标跳到哪，当前的参数即可通过加减键修改

2. 设计的电路，完成相应器件的选择和参数计算，制作 Arduino UNO 扩展板。

3. 编制与调试多功能数字时钟程序。

4. 将制作的 Arduino UNO 扩展板与 Arduino UNO 板组装后，进行系统联调。

三、 实验设计方案

本项目使用 Arduino UNO 设计多功能数字时钟。硬件框图如 Figure 1 所示，使用出的模块在 Table 1 中列出，下面对各部分模块进行说明。

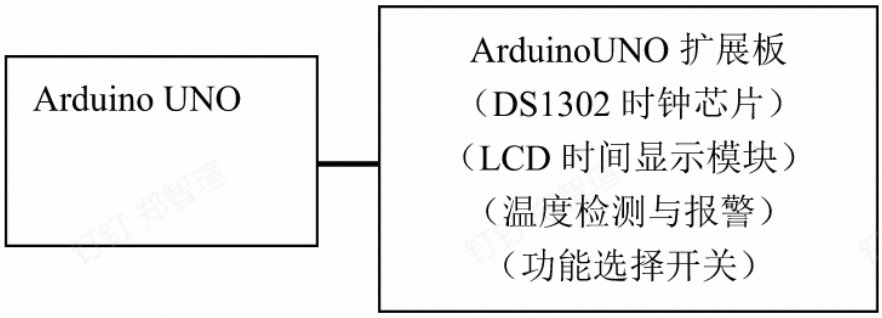


Figure 1: 多功能数字时钟框图

编号	仪器用具名称	备注
1	PC 电脑	使用 Altium Designer Winter 09 进行设计
2	Arduino UNO	使用 PlatformIO 进行开发
3	DS1302 时钟模块	使用 DS1302 库
4	LCD1602 液晶显示屏	使用 LCD1602 库
6	LM35 温度传感器	–
5	无源蜂鸣器	
7	按键	–
8	电阻、排针若干	–

Table 1: 实验设备列表

1. DS1302 时钟芯片

DS1302 是美国 DALLAS 公司推出的一种高性能、低功耗的实时时钟芯片，附加 31 字节静态 RAM，采用 SPI 三线接口与 CPU 进行同步通信，并可采用突发方式一次传送多个字节的时钟信号和 RAM 数据。实时时钟可提供秒、分、时、日、星期、月和年，一个月小与 31 天时可以自动调整，且具有闰年补偿功能。工作电压宽达 2.5～5.5V。采用双电源供电（主电源和备用电源），可设置备用电源充电方式，提供了对后背电源进行涓细电流充电的能力。



Figure 2: DS1032 模块图

2. LCD 显示模块

本次实验使用 Arduino UNO 直接驱动 1602 液晶显示字母和数字，LCD 显示模块有一个优势就是将驱动电路集成在模块中。LCD 显示模块具有标准，不同的生产商所生产的大多数模块都可以按相同方法来使用。1602 液晶在应用中非常广泛，最初的 1602 液晶使用的是 HD44780 控制器，现在各个厂家的 1602 模块基本上都是采用了与之兼容的 IC，所以特性上基本都是一致的。



Figure 3: LCD1602 模块图

3. 温度传感器

温度传感器可采用 LM35，LM35 是由 National Semiconductor 所生产的温度传感器，其输出电压为摄氏温标。LM35 是一种得到广泛使用的温度传感器。其电源供应模式有单电源与正负双电源两种，其引脚正负双电源的供电模式可提供负温度的量测；两种接法的静止电流-温度关系，在静止温度中自热效应低 (0.08℃)，单电源模式在 25℃下静止电流约 50μA，工作电压较宽，可在 4—20V 的供电电压范围内正常工作非常省电。



Figure 4: LM35 模块图

4. 无源蜂鸣器

无源蜂鸣器需要 Arduino 提供特定频率的脉冲信号，并将方波信号输入谐振装置转换为声音信号输出才能发声，可以通过改变信号频率来调节音调。



Figure 5: 无源蜂鸣器模块图

四、 硬件设计

1. 参考原理图

本设计实验主要参考以下设计示例图：

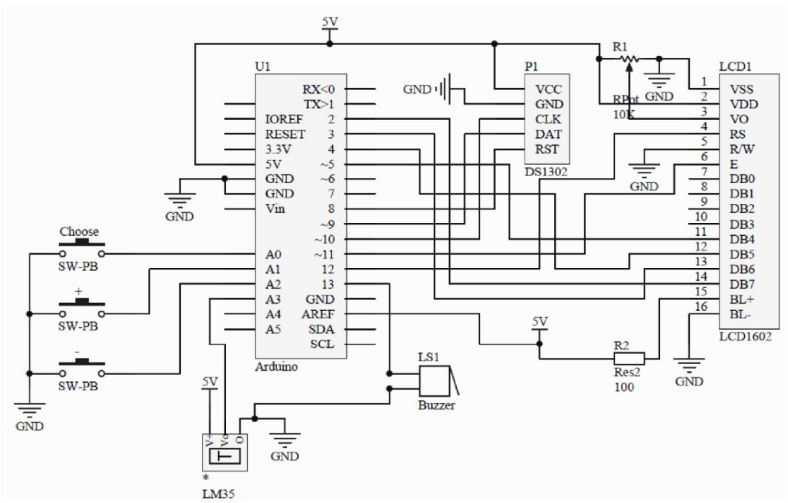


Figure 6: 参考原理图

2. 原理图

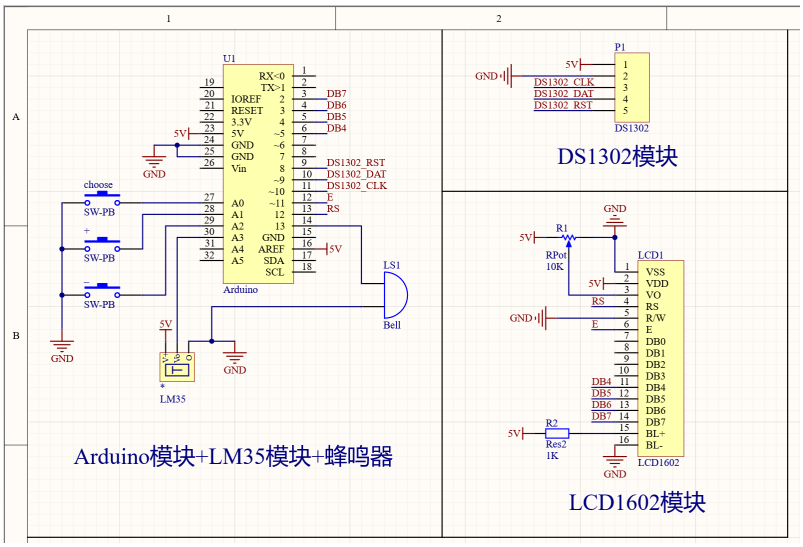


Figure 7: 原理图

3. PCB 图

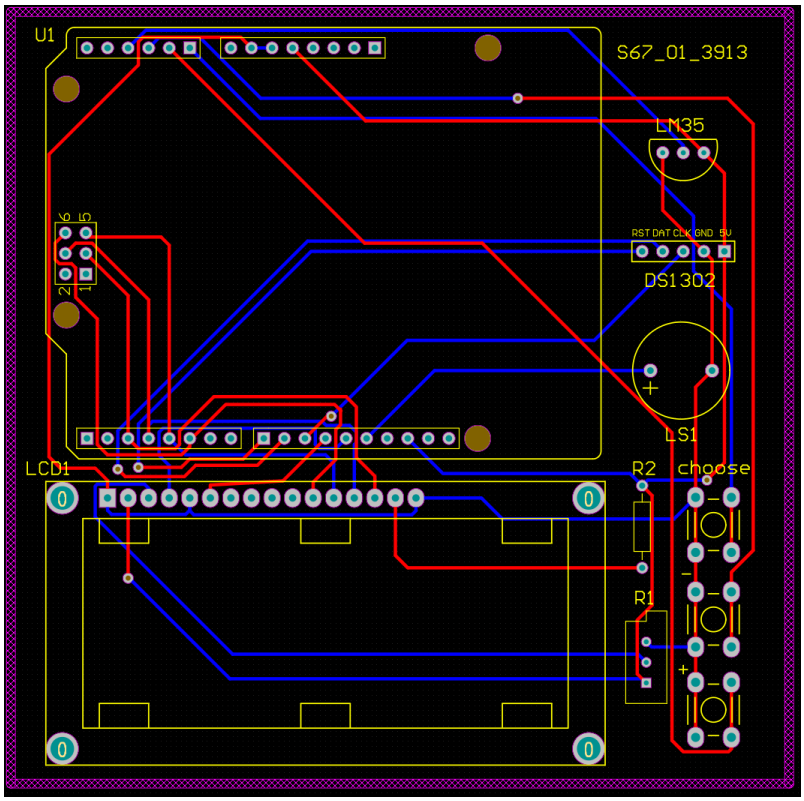


Figure 8: PCB 图

4. PCB 实物图

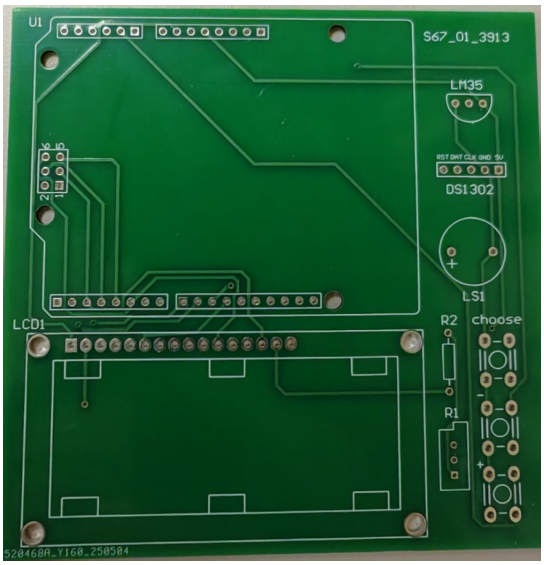


Figure 9: PCB 正面实物图

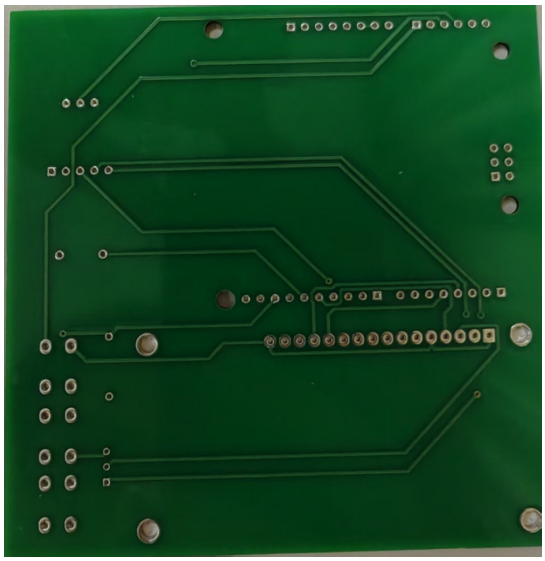


Figure 10: PCB 反面实物图

五、 软件设计

1. 接线设计

模块	接线关系
LCD1602	VSS → GND VDD → 5V RS → UNO.12 R/W → GND E → UNO.11 DB4 → UNO.5 DB5 → UNO.4 DB6 → UNO.3 DB7 → UNO.2
按键	Choose → UNO.A0 + → UNO.A1 - → UNO.A2
DS1302	CLK → UNO.10 DAT → UNO.9 RST → UNO.8
LM35	Vo → UNO.A3
蜂鸣器	Buzzer → UNO.13

Table 2: 接线关系表

2. 代码设计

(1) 闹钟报警优化

在所给程序的基础上，我们进行了优化，首先，我们加入了 ToneConfig.h 文件，保存浙大校歌《大不自多》的音符数据，以下是 ToneConfig.h 文件的主要内容：

Listing 1: Arduino 数字时钟音符配置文件

```
#ifndef TONE_CONFIG_H
#define TONE_CONFIG_H

#define NOTE_B0 31
#define NOTE_C1 33
#define NOTE_CS1 35
/* 省略部分宏定义 */
#define NOTE_D8 4699
#define NOTE_DS8 4978
#define REST   0

// 每个音符持续时间宏(毫秒)
#define BASE_DURATION 250

const int zheDaSong[] = {
  NOTE_E5, NOTE_E5, NOTE_E5,
  NOTE_G5, NOTE_C5,
  NOTE_D5, NOTE_F5,
  /* 后续省略 */
};

// 每个音符持续时间(毫秒)
const int noteDurations[] = {
  BASE_DURATION*2, BASE_DURATION*2, BASE_DURATION*2,
  BASE_DURATION*4, BASE_DURATION*2,
  BASE_DURATION*4, BASE_DURATION*2,
  /* 后续省略 */
};

// 音乐长度
const int songLength = sizeof(zheDaSong) / sizeof(int);

#endif
```

其次，我们优化了原有的闹钟报警代码，使其非阻塞式运行，避免了在播放音符时阻塞主循环，从而使得其他功能（如温度检测、时间显示等）可以正常运行。以下是优化后的代码片段：

Listing 2: Arduino 数字时钟报警优化

```
// 闹钟音乐控制变量
bool alarmActive = false;      // 闹钟是否激活
int currentNoteIndex = 0;      // 当前播放的音符索引
unsigned long lastNoteTime = 0; // 上次音符切换的时间
unsigned long currentNoteDuration; // 当前音符持续时间

/** 正点蜂鸣（非阻塞）*/
void Point_Time_Alarm(){
  if(minute == 0 && second == 0 && !hourAlarmActive && !alarmActive) {
    // 激活整点报时
    hourAlarmActive = true;
    hourAlarmStartTime = millis();
    alarmActive = true;
    currentNoteIndex = 1;
    lastNoteTime = millis();
    currentNoteDuration = noteDurations[0];
    isPlayingTone = true;
    if (zheDaSong[0] != REST) {
      tone(Tone, zheDaSong[0]);
    }
  }
}

/** 闹钟指定时间蜂鸣（非阻塞） 与前类似，省略*/
/** 超过指定温度报警（非阻塞） 与前类似，省略 */

void playZheDaSong() {
  unsigned long currentTime = millis();

  // 如果没有激活报警，直接返回
  if (!alarmActive) {
    return;
  }

  if (screenOff && alarmActive) {
    turnOnScreen();
  }

  // 检查当前音符是否已经播放足够长的时间
  if (isPlayingTone && currentTime - lastNoteTime >= currentNoteDuration - noteSpacing) {
    // 音符播放完毕，进入静音间隔
```

```
    noTone(Tone);
    isPlayingTone = false;
    lastNoteTime = currentTime;
  }
  else if (!isPlayingTone && currentTime - lastNoteTime >= noteSpacing) {
    // 静音间隔结束，播放下一个音符
    isPlayingTone = true;

    // 移动到下一个音符
    if (currentNoteIndex < songLength) {
      // 如果是休止符，保持静音
      if (zheDaSong[currentNoteIndex] != REST) {
        tone(Tone, zheDaSong[currentNoteIndex]);
      }

      // 设置音符持续时间
      currentNoteDuration = noteDurations[currentNoteIndex];
      lastNoteTime = currentTime;
      currentNoteIndex++;
    } else {
      // 播放完毕，重置变量
      alarmActive = false;
      currentNoteIndex = 0;
      isPlayingTone = true;
      noTone(Tone);
    }
  }
}
```

(2) 熄屏省电优化

在实际生活中，功率是一个需要关注的问题，所以我们设计了一个电子钟熄屏省电模式功能。当设备长时间不使用时，电子钟会自动进入省电模式，这能够有效降低电子钟的功耗。

在省电状态下，系统会关闭 LCD 的显示，可以减少在不需要频繁查看时间时的能源浪费。在省电模式下，电子钟仍能进行报警，同时通过简单的按键触碰操作可以随时恢复到正常工作模式，以下是省电模式的实现代码：

Listing 3: Arduino 数字时钟省电优化

```
// 熄屏省电模式变量
unsigned long lastActionTime = 0; // 上次操作时间
bool screenOff = false;          // 屏幕状态
const unsigned long SCREEN_TIMEOUT = 10000; // 屏幕超时时间(10秒)

/** 熄屏省电模式 */
void turnOffScreen() {
  if (!screenOff) {
    lcd.noDisplay();             // 关闭LCD显示
    screenOff = true;
    noTone(Tone);                // 停止蜂鸣器
    // 保留闹钟功能，但暂时不播放音乐
  }
}

/** 唤醒屏幕 */
void turnOnScreen() {
  if (screenOff) {
    lcd.display();               // 重新开启LCD显示
    screenOff = false;
    lastActionTime = millis();   // 重置操作时间
  }
}

/** 更新用户操作时间 */
void updateActionTime() {
  lastActionTime = millis();
  turnOnScreen(); // 如果屏幕关闭，则唤醒屏幕
}
```

六、 调试

1. 元器件安放问题

在调试时，我们首先发现，LCD1602 的实物比 PCB 上显示的封装要大，但此时电位器已经焊接上，所以只能采取调一下电位器，插一下 LCD1602 的方法来调节字符的明暗对比度，比较麻烦

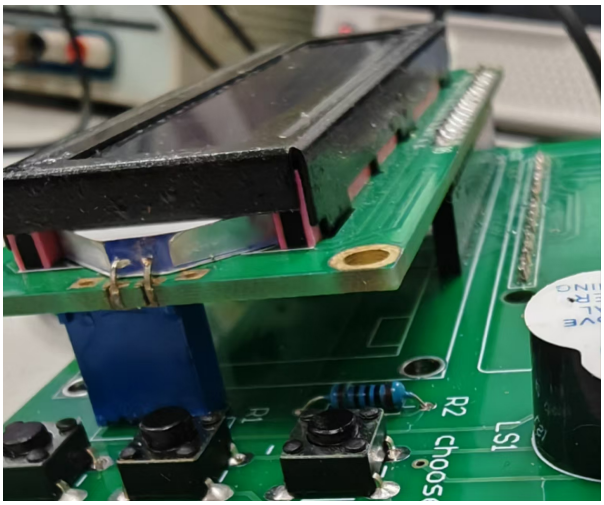


Figure 11: 元器件安放问题示意图

在继续调试时，我们发现，蜂鸣器不能发声，经过排查，发现不是软件问题，后续发现是蜂鸣器的正口接上 Arduino 板子的插线外缘，被强制接地导致不能发声，后续采取垫一张小纸片进行隔离，成功解决

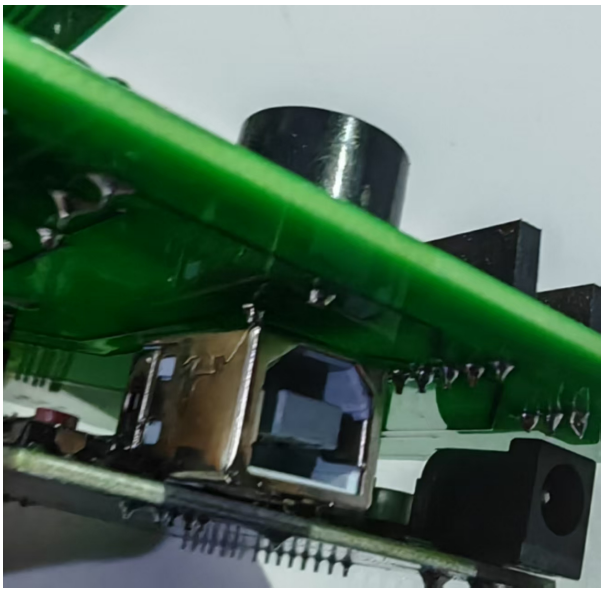


Figure 12: 元器件安放问题示意图

【总结】：以后需要吸取经验，在设计 PCB 时要注意实物的大小与点位，防止以上现象的发生。

2. 闹钟阻塞运行问题

我们发现初始的 Clock_ALram 函数里使用了 delay 函数来控制响铃时间，这种阻塞式运行方法会使得其他功能无法正常运行，所以我们采取了类似中断触发的方法：在每次循环中检查当前时间是否到达闹钟时间，如果到达，则置布尔变量 alarmActive 为 true 以模拟中断开始，并在播放音乐时不使用 delay 函数，而是使用 millis() 函数来控制音符的播放时间，这样可以使得其他功能（如温度检测、时间显示等）可以正常运行。

同时，经过如此处理，可以便捷的使用接口函数 stopAlarm() 来实现按下按钮即停止闹钟响铃。类似的，我们更改了 Point_Time_Alarm() 与 Temperatures_Alarm() 函数，不在此一一列出

Listing 4: Arduino 数字时钟闹钟阻塞运行处理

```
void Clock_Alarm(){
  if(hour == alarm_hour && minute == alarm_minute && second == alarm_second && !alarmActive) {
    // 激活闹钟
    alarmActive = true;
    /* 省略其他代码 */
    }
}

void playZheDaSong() {
  unsigned long currentTime = millis();

  /* 省略其他代码 */
  else if (!isPlayingTone && currentTime - lastNoteTime >= noteSpacing) {
    // 静音间隔结束，播放下一个音符
    isPlayingTone = true;
  }
  /* 省略其他代码 */
}

void stopAlarm() {
  alarmActive = false;
  /* 省略其他代码 */
  noTone(Tone);
}
```

七、 功能实现

1. 显示时间、日期、星期和温度



Figure 13: 显示时间、日期、星期和温度示意图

2. 按下 choose 按键显示光标进入设置时间模式

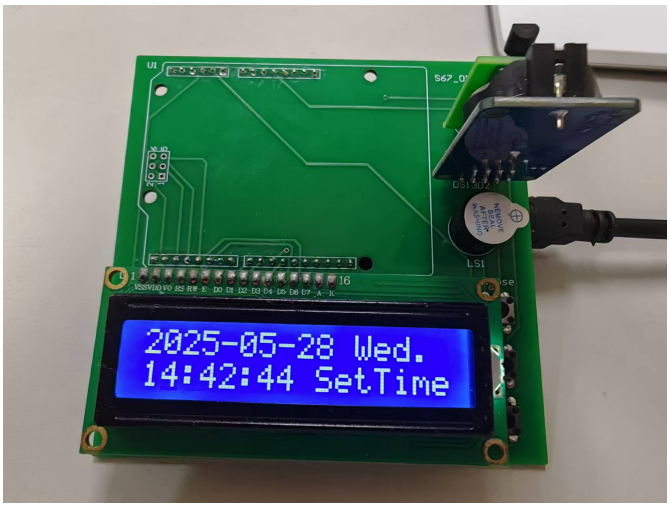


Figure 14: 显示时间、日期、星期和温度示意图

3. 同时按下“-”和“+”进入设置闹钟和报警温度模式



Figure 15: 按下“-”和“+”进入设置闹钟和报警温度模式示意图

4. 长时间未使用息屏功能

长时间不使用会进行熄屏，实现省电的目标，息屏状态下按任意按钮即可恢复显示

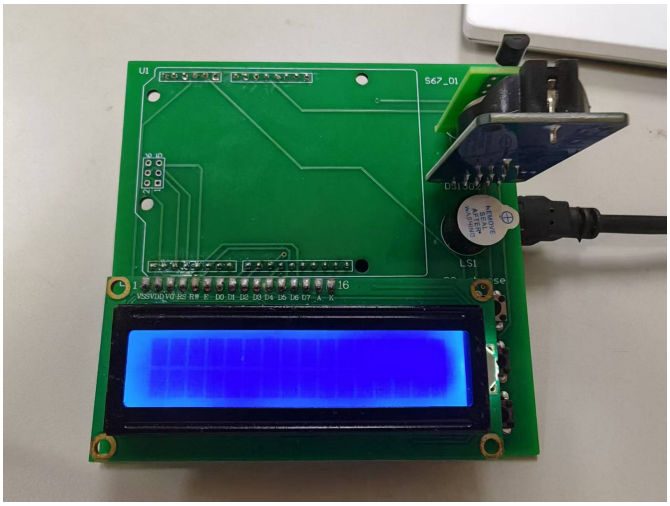


Figure 16: 长时间未使用息屏功能示意图

5. 播放/停止音乐功能

：当高于报警温度/到闹钟设定时间/整点时，播放浙大校歌，并且实现了按下任意键后音乐停止

八、 实验总结与心得

1. 实验总结

通过本次 Arduino 多功能数字时钟的设计与实现，我全面掌握了嵌入式系统开发的完整流程, 获得了多种能力的提升：

(1) 硬件设计能力

成功设计并制作了 Arduino 扩展板，实现 DS1302 时钟模块、LCD1602 显示屏、LM35 温度传感器等关键元件的集成

(2) 软件编程突破

实现非阻塞式报警系统（整点报时、闹钟、温度报警）创新开发省电模式（自动熄屏 + 按键唤醒）完成浙大校歌《大不自多》的音乐编解码与播放

(3) 系统调试经验

排查出了 LCD 尺寸不匹配、蜂鸣器接地异常等硬件问题

(4) 小结

通过这一系列密集且细致的实践，我掌握了如何将抽象的电路原理转化为可实际加工和测试的 PCB 版图，并在过程中深刻理解了诸多设计原则的实际应用。这种经历不仅增强了我的工程实践能力，也为我今后在电子工程领域的专业发展奠定了坚实基础。未来，我将以更加系统化、专业化的视角应对各类电路设计挑战，不断提升自身的技术水平与创新能力。

2. 实验心得

在本次实验中，我深刻体会到了电子电路设计的复杂性和挑战性。通过实际操作，我不仅巩固了之前学过的理论知识，还学会了如何将这些知识应用到实际的电路设计中。在设计过程中，我遇到了许多问题，比如元器件的选择、PCB 布局的优化等，这些都需要我不断地学习和尝试。通过查阅资料和请教老师，我逐渐克服了这些困难，完成了实验任务。

总的来说，这次实验让我对电子电路设计有了更深入的理解，也提高了我的动手能力和解决问题的能力。我认识到，电子电路设计不仅仅是一个技术活，更是一个需要创造力和耐心的过程。未来，我希望能在这个领域继续学习和探索，不断提升自己的专业技能。

九、 完整代码

Listing 5: ”ToneConfig.h”

```
#ifndef TONE_CONFIG_H
#define TONE_CONFIG_H

// 音符频率定义
#define NOTE_B0 31
#define NOTE_C1 33
#define NOTE_CS1 35
#define NOTE_D1 37
#define NOTE_DS1 39
#define NOTE_E1 41
#define NOTE_F1 44
#define NOTE_FS1 46
#define NOTE_G1 49
#define NOTE_GS1 52
#define NOTE_A1 55
#define NOTE_AS1 58
#define NOTE_B1 62
#define NOTE_C2 65
#define NOTE_CS2 69
#define NOTE_D2 73
#define NOTE_DS2 78
#define NOTE_E2 82
#define NOTE_F2 87
#define NOTE_FS2 93
#define NOTE_G2 98
#define NOTE_GS2 104
#define NOTE_A2 110
#define NOTE_AS2 117
#define NOTE_B2 123
#define NOTE_C3 131
#define NOTE_CS3 139
#define NOTE_D3 147
#define NOTE_DS3 156
#define NOTE_E3 165
#define NOTE_F3 175
#define NOTE_FS3 185
#define NOTE_G3 196
#define NOTE_GS3 208
#define NOTE_A3 220
#define NOTE_AS3 233
#define NOTE_B3 247
```

```
42 #define NOTE_C4 262
43 #define NOTE_CS4 277
44 #define NOTE_D4 294
45 #define NOTE_DS4 311
46 #define NOTE_E4 330
47 #define NOTE_F4 349
48 #define NOTE_FS4 370
49 #define NOTE_G4 392
50 #define NOTE_GS4 415
51 #define NOTE_A4 440
52 #define NOTE_AS4 466
53 #define NOTE_B4 494
54 #define NOTE_C5 523
55 #define NOTE_CS5 554
56 #define NOTE_D5 587
57 #define NOTE_DS5 622
58 #define NOTE_E5 659
59 #define NOTE_F5 698
60 #define NOTE_FS5 740
61 #define NOTE_G5 784
62 #define NOTE_GS5 831
63 #define NOTE_A5 880
64 #define NOTE_AS5 932
65 #define NOTE_B5 988
66 #define NOTE_C6 1047
67 #define NOTE_CS6 1109
68 #define NOTE_D6 1175
69 #define NOTE_DS6 1245
70 #define NOTE_E6 1319
71 #define NOTE_F6 1397
72 #define NOTE_FS6 1480
73 #define NOTE_G6 1568
74 #define NOTE_GS6 1661
75 #define NOTE_A6 1760
76 #define NOTE_AS6 1865
77 #define NOTE_B6 1976
78 #define NOTE_C7 2093
79 #define NOTE_CS7 2217
80 #define NOTE_D7 2349
81 #define NOTE_DS7 2489
82 #define NOTE_E7 2637
83 #define NOTE_F7 2794
84 #define NOTE_FS7 2960
85 #define NOTE_G7 3136
86 #define NOTE_GS7 3322
```

```
#define NOTE_A7 3520
#define NOTE_AS7 3729
#define NOTE_B7 3951
#define NOTE_C8 4186
#define NOTE_CS8 4435
#define NOTE_D8 4699
#define NOTE_DS8 4978
#define REST   0

#define BASE_DURATION 250

// 浙大校歌《大不自多》音符序列
const int zheDaSong[] = {
  // 第一段
  NOTE_E5, NOTE_E5, NOTE_E5,
  NOTE_G5, NOTE_C5,
  NOTE_D5, NOTE_F5,
  NOTE_E5, NOTE_G5,
  NOTE_A5, NOTE_G5, NOTE_F5,
  NOTE_A5, REST,

  NOTE_A5, NOTE_B5, NOTE_A5,
  NOTE_G5, NOTE_G5,
  NOTE_C6, NOTE_B5, NOTE_A5,
  NOTE_G5, NOTE_F5, NOTE_E5,
  NOTE_A5, NOTE_D5, NOTE_E5,
  NOTE_D5, REST
};

// 每个音符持续时间(毫秒)
const int noteDurations[] = {
  // 第一段
  BASE_DURATION*2, BASE_DURATION*2, BASE_DURATION*2,
  BASE_DURATION*4, BASE_DURATION*2,
  BASE_DURATION*4, BASE_DURATION*2,
  BASE_DURATION*4, BASE_DURATION*2,
  BASE_DURATION*2, BASE_DURATION*2, BASE_DURATION*2,
  BASE_DURATION*4, BASE_DURATION*2,

  BASE_DURATION*2, BASE_DURATION*2, BASE_DURATION*2,
  BASE_DURATION*4, BASE_DURATION*2,
  BASE_DURATION*4, BASE_DURATION*1, BASE_DURATION*1,
  BASE_DURATION*3, BASE_DURATION*1, BASE_DURATION*2,
  BASE_DURATION*2, BASE_DURATION*2, BASE_DURATION*2,
  BASE_DURATION*4, BASE_DURATION*2
```

```
};

// 音乐长度
const int songLength = sizeof(zheDaSong) / sizeof(int);

#endif
```

Listing 6: ”main.cpp”

```
#include <DS1302.h>
#include <LiquidCrystal.h> //LCD1602 显示头文件
#include "ToneConfig.h"

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
#define choose A0 //选择端口
#define add A1 //加
#define minus A2 //减
#define Tone 13 //蜂鸣器端口
uint8_t CE_PIN = 8; //DS1302 RST 端口
uint8_t IO_PIN = 9; //DS1302 DAT 端口
uint8_t SCLK_PIN = 10; //DS1302 CLK 端口
DS1302 rtc(CE_PIN, IO_PIN, SCLK_PIN); //创建 DS1302 对象
unsigned long seconds;
int s = 0, m = 0, h = 0, d = 0, mon = 0, y = 0; //时间进位
int second = 0, minute = 0, hour = 0, day = 0, month = 0, year = 0; //当前时间
int SECOND = 0, MINUTE = 0, HOUR = 0, DAY = 0, MONTH = 0, YEAR = 0; //初始时间
int chose = 0, alarm_choose = 0 ,ButtonDelay = 20, frequence = 2093;
int alarm_hour = 7, alarm_minute = 30, alarm_second = 0; //闹钟时间
float Temperatures, Temp_Alarm = 30 ;

/* 新增 */

// 熄屏省电模式变量
unsigned long lastActionTime = 0; // 上次操作时间
bool screenOff = false;         // 屏幕状态
const unsigned long SCREEN_TIMEOUT = 10000; // 屏幕超时时间(10秒)

// 闹钟音乐控制变量
bool alarmActive = false;      // 闹钟是否激活
int currentNoteIndex = 0;      // 当前播放的音符索引
unsigned long lastNoteTime = 0; // 上次音符切换的时间
unsigned long currentNoteDuration; // 当前音符持续时间

// 温度报警控制变量
bool tempAlarmActive = false;
unsigned long lastTempAlarmTime = 0;
```

```
38
39 // 整点报时控制变量
40 bool hourAlarmActive = false;
41 unsigned long hourAlarmStartTime = 0;
42
43 // 添加新的状态变量
44 bool isPlayingTone = true;     // 是否正在播放音符(而非间隔)
45 unsigned long noteSpacing = 50; // 音符之间的间隔时间(毫秒)
46
47
48 /** 正点蜂鸣（非阻塞）*/
49 void Point_Time_Alarm(){
50   if(minute == 0 && second == 0 && !hourAlarmActive && !alarmActive) {
51     // 激活整点报时
52     hourAlarmActive = true;
53     hourAlarmStartTime = millis();
54     alarmActive = true;
55     currentNoteIndex = 1; // 修改为1，因为已经播放第一个音符
56     lastNoteTime = millis();
57     currentNoteDuration = noteDurations[0];
58     isPlayingTone = true;
59     // 立即播放第一个音符
60     if (zheDaSong[0] != REST) {
61       tone(Tone, zheDaSong[0]);
62     }
63   }
64 }
65
66 /** 闹钟 指定时间蜂鸣（非阻塞）*/
67 void Clock_Alarm(){
68   if(hour == alarm_hour && minute == alarm_minute && second == alarm_second && !alarmActive) {
69     // 激活闹钟
70     alarmActive = true;
71     currentNoteIndex = 1;
72     lastNoteTime = millis();
73     currentNoteDuration = noteDurations[0];
74     isPlayingTone = true;
75     // 立即播放第一个音符
76     if (zheDaSong[0] != REST) {
77       tone(Tone, zheDaSong[0]);
78     }
79   }
80 }
81
82 /** 超过指定温度报警（非阻塞）*/
```

```
83  void Temperatures_Alarm(){
84    unsigned long currentTime = millis();
85
86    // 检查温度，如果超过设定值且当前没有温度报警
87    if(Temperatures >= Temp_Alarm && !tempAlarmActive && !alarmActive) {
88      tempAlarmActive = true;
89      lastTempAlarmTime = currentTime;
90
91      // 激活报警播放
92      alarmActive = true;
93      currentNoteIndex = 1;
94      lastNoteTime = currentTime;
95      currentNoteDuration = noteDurations[0];
96      isPlayingTone = true;
97      // 立即播放第一个音符
98      if (zheDaSong[0] != REST) {
99        tone(Tone, zheDaSong[0]);
100     }
101   }
102 }
103
104 /** 熄屏省电模式 */
105 void turnOffScreen() {
106   if (!screenOff) {
107     lcd.noDisplay();              // 关闭LCD显示
108     screenOff = true;
109     noTone(Tone);                 // 停止蜂鸣器
110     // 保留闹钟功能，但暂时不播放音乐
111   }
112 }
113
114 /** 唤醒屏幕 */
115 void turnOnScreen() {
116   if (screenOff) {
117     lcd.display();                // 重新开启LCD显示
118     screenOff = false;
119     lastActionTime = millis();    // 重置操作时间
120   }
121 }
122
123 /** 更新用户操作时间 */
124 void updateActionTime() {
125   lastActionTime = millis();
126   turnOnScreen(); // 如果屏幕关闭，则唤醒屏幕
127 }
```

```
128
129 void stopAlarm() {
130   alarmActive = false;
131   tempAlarmActive = false;
132   hourAlarmActive = false;
133   isPlayingTone = true;
134   currentNoteIndex = 0;
135   noTone(Tone); // 立即停止蜂鸣器发声
136 }
137
138 /** 检测按键活动 */
139 bool checkButtonActivity() {
140   // 检查所有按键
141   if (digitalRead(choose) == LOW || digitalRead(add) == LOW || digitalRead(minus) == LOW) {
142     updateActionTime();
143   if (alarmActive || tempAlarmActive || hourAlarmActive) {
144     stopAlarm();
145     }
146     return true;
147   }
148   return false;
149 }
150
151 /** 非阻塞播放浙大校歌 */
152 void playZheDaSong() {
153   unsigned long currentTime = millis();
154
155   // 如果没有激活报警，直接返回
156   if (!alarmActive) {
157     return;
158   }
159
160   if (screenOff && alarmActive) {
161     turnOnScreen();
162   }
163
164   // 检查当前音符是否已经播放足够长的时间
165   if (isPlayingTone && currentTime - lastNoteTime >= currentNoteDuration - noteSpacing) {
166     // 音符播放完毕，进入静音间隔
167     noTone(Tone);
168     isPlayingTone = false;
169     lastNoteTime = currentTime;
170   }
171   else if (!isPlayingTone && currentTime - lastNoteTime >= noteSpacing) {
172     // 静音间隔结束，播放下一个音符
```

```
    isPlayingTone = true;

    // 移动到下一个音符
    if (currentNoteIndex < songLength) {
      // 如果是休止符，保持静音
      if (zheDaSong[currentNoteIndex] != REST) {
        tone(Tone, zheDaSong[currentNoteIndex]);
      }

      // 设置音符持续时间
      currentNoteDuration = noteDurations[currentNoteIndex];
      lastNoteTime = currentTime;
      currentNoteIndex++;
    } else {
      // 播放完毕，重置变量
      alarmActive = false;
      currentNoteIndex = 0;
      isPlayingTone = true;
      noTone(Tone);
    }
  }
}

/* 新增 */

/** 格式化输出 */
void FormatDisplay(int col, int row,int num){
    lcd.setCursor(col, row);
    if(num < 10) lcd.print("0");
    lcd.print(num);
}

/** 计算时间 */
void time() {
    second = (SECOND + seconds) % 60; //计算秒
    m = (SECOND + seconds) / 60; //分钟进位
    FormatDisplay(6,1,second);
    minute = (MINUTE + m) % 60; //计算分钟
    h = (MINUTE + m) / 60; //小时进位
    FormatDisplay(3,1,minute);
    hour = (HOUR + h) % 24; //计算小时
    d = (HOUR + h) / 24; //天数进位
    FormatDisplay(0,1,hour);
    lcd.setCursor(2, 1);
    lcd.print(":");
```

```
218     lcd.setCursor(5, 1);
219     lcd.print(":");
220 }
221 
222 /** 根据年月计算当月天数 */
223 int Days(int year, int month){
224     int days = 0;
225     if (month != 2){
226         switch(month){
227             case 1: case 3: case 5: case 7: case 8: case 10: case 12:
228                 days = 31;
229                 break;
230             case 4: case 6: case 9: case 11:
231                 days = 30;
232                 break;
233         }
234     }else{ //闰年
235         if(year % 4 == 0 && year % 100 != 0 || year % 400 == 0){
236             days = 29;
237         }
238         else{
239             days = 28;
240         }
241     }
242     return days;
243 }
244 
245 /** 计算当月天数 */
246 void Day(){
247     int days = Days(year,month);
248     int days_up;
249     if(month == 1){
250         days_up = Days(year - 1, 12);
251     }
252     else{
253         days_up = Days(year, month - 1);
254     }
255     day = (DAY + d) % days;
256     if(day == 0){
257         day = days;
258     }
259     if((DAY + d) == days + 1 ){
260         DAY -= days;
261         mon++;
262     }
```

```
263     if((DAY + d) == 0){
264         DAY += days_up;
265         mon--;
266     }
267     FormatDisplay(8,0,day);
268 }
269 
270 /** 计算月份 */
271 void Month(){
272     month = (MONTH + mon) % 12;
273     if(month == 0){
274         month = 12;
275     }
276     y = (MONTH + mon - 1) / 12;
277     FormatDisplay(5,0,month);
278     lcd.setCursor(7, 0);
279     lcd.print('-');
280 }
281 
282 /** 计算年份 */
283 void Year(){
284     year = ( YEAR + y ) % 9999;
285     if(year == 0){
286         year = 9999;
287     }
288     lcd.setCursor(0, 0);
289     if(year < 1000){
290         lcd.print("0");
291     }
292     if(year < 100){
293         lcd.print("0");
294     }
295     if(year < 10){
296         lcd.print("0");
297     }
298     lcd.print(year);
299     lcd.setCursor(4, 0);
300     lcd.print('-');
301 }
302 
303 /** 根据年月日计算星期几 */
304 void Week(int y,int m, int d){
305     if(m == 1){
306         m = 13;
307     }
```

```
308     if(m == 2){
309         m = 14;
310     }
311     int week = (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)%7+1;
312     String weekstr = "";
313     switch(week){
314         case 1: weekstr = "Mon. "; break;
315         case 2: weekstr = "Tues. "; break;
316         case 3: weekstr = "Wed. "; break;
317         case 4: weekstr = "Thur. "; break;
318         case 5: weekstr = "Fri. "; break;
319         case 6: weekstr = "Sat. "; break;
320         case 7: weekstr = "Sun. "; break;
321     }
322     lcd.setCursor(11, 0);
323     lcd.print(weekstr);
324 }
325 
326 /** 显示时间、日期、星期 */
327 void Display() {
328     time();
329     Day();
330     Month();
331     Year();
332     Week(year,month,day);
333 }
334 
335 /** 显示光标 */
336 void DisplayCursor(int rol, int row) {
337     lcd.setCursor(rol, row);
338     lcd.cursor();
339     delay(100);
340     lcd.noCursor();
341     delay(100);
342 }
343 
344 /** 设置初始时间 */
345 void set(int y, int mon, int d, int h, int m, int s){
346     YEAR = y;
347     MONTH = mon;
348     DAY = d;
349     HOUR = h;
350     MINUTE = m;
351     SECOND = s;
352 }
```

```
353
354 /** 通过按键设置时间 */
355 void Set_Time(int rol, int row, int &Time){
356     DisplayCursor(rol, row);
357     if(digitalRead(add) == LOW){
358         delay(ButtonDelay);
359         if(digitalRead(add) == LOW){
360             Time ++;
361         }
362         Display();
363     }
364     if(digitalRead(minus) == LOW){
365         delay(ButtonDelay);
366         if(digitalRead(minus) == LOW){
367             Time --;
368         }
369         Display();
370     }
371 }
372
373 /** 按键选择 */
374 void Set_Clock(){
375     if(digitalRead(choose)==LOW){
376         updateActionTime();
377         if (alarmActive || tempAlarmActive || hourAlarmActive) {
378           stopAlarm();
379           delay(100); // 短暂延时，避免立即进入设置模式
380           return;     // 第一次按键仅用于停止报警
381         }
382         lcd.setCursor(9, 1);
383         lcd.print("SetTime");
384         while(1){
385             if(digitalRead(choose) == LOW){
386                 delay(ButtonDelay);
387                 if(digitalRead(choose) ==LOW){
388                     chose++;
389                 }
390             }
391             seconds = millis()/1000;
392             Display();
393             if(chose == 1){
394                 Set_Time(1, 1, HOUR); //SetHour
395             }else if(chose == 2){
396                 Set_Time(4, 1, MINUTE); //SetMinute
397             }else if(chose == 3){
```

```
398                 Set_Time(7, 1, SECOND); //SetSecond
399             }else if(chose == 4){
400                 Set_Time(9, 0, DAY); //SetDay
401             }else if(chose == 5){
402                 Set_Time(6, 0, MONTH); // SetMonth
403             }else if(chose == 6){
404                 Set_Time(3, 0, YEAR); //SetYear
405             }else if(chose >= 7) {
406                 updateActionTime();
407                 chose = 0;
408                 break;
409             }
410         }
411         delay(150);//新增（2024）
412     }
413 }
414 
415 /** 设置闹钟小时 */
416 void Set_Alarm_Hour(){
417     DisplayCursor(1, 1);
418     if(digitalRead(add) == LOW){
419         delay(ButtonDelay);
420         if(digitalRead(add) == LOW){
421             alarm_hour ++;
422             if(alarm_hour == 24){
423                 alarm_hour = 0;
424             }
425             FormatDisplay(0,1,alarm_hour);
426         }
427     }
428     if(digitalRead(minus) == LOW){
429         delay(ButtonDelay);
430         if(digitalRead(minus) == LOW){
431             alarm_hour --;
432             if(alarm_hour == -1){
433                 alarm_hour = 23;
434             }
435             FormatDisplay(0,1,alarm_hour);
436         }
437     }
438 }
439 
440 /** 设置闹钟分钟 */
441 void Set_Alarm_Minute(){
442     DisplayCursor(4, 1);
```

```
443     if(digitalRead(add) == LOW) {
444         delay(ButtonDelay);
445         if(digitalRead(add) == LOW){
446             alarm_minute ++;
447             if(alarm_minute == 60){
448                 alarm_minute = 0;
449             }
450             FormatDisplay(3,1,alarm_minute);
451         }
452     }
453     if(digitalRead(minus) == LOW){
454         delay(ButtonDelay);
455         if(digitalRead(minus) == LOW){
456             alarm_minute --;
457             if(alarm_minute == -1){
458                 alarm_minute = 59;
459             }
460             FormatDisplay(3,1,alarm_minute);
461         }
462     }
463 }
464 
465 /** 设置报警温度 */
466 void Set_Alarm_Temp(){
467     DisplayCursor(10, 1);
468     if(digitalRead(add) == LOW) {
469         delay(ButtonDelay);
470         if(digitalRead(add) == LOW){
471             Temp_Alarm ++;
472         }
473     }
474     if(digitalRead(minus) == LOW){
475         delay(ButtonDelay);
476         if(digitalRead(minus) == LOW){
477             Temp_Alarm --;
478         }
479     }
480 }
481 
482 /** 进入报警设置 */
483 void Set_Alarm(){
484     if(digitalRead(add) == LOW && digitalRead(minus) == LOW){
485         if (alarmActive || tempAlarmActive || hourAlarmActive) {
486           stopAlarm();
487           delay(100); // 短暂延时，避免立即进入设置模式
```

```
488           return;     // 仅用于停止报警
489         }
490         alarm_hour = hour;
491         alarm_minute = minute;
492         lcd.setCursor(0, 0);
493         lcd.print("set alarm ");
494         lcd.setCursor(6, 1);
495         lcd.print("00"); //闹钟秒数
496         while(1){
497             if(digitalRead(choose) == LOW){
498                 delay(ButtonDelay);
499                 if(digitalRead(choose) == LOW){
500                     alarm_choose++;
501                 }
502             }
503             lcd.setCursor(9, 1);
504             lcd.print(Temp_Alarm);
505             lcd.setCursor(14, 1);
506             lcd.print((char)223); //显示 o 符号
507             lcd.setCursor(15, 1);
508             lcd.print("C"); //显示字母 C
509             if(alarm_choose == 1){
510                 Set_Alarm_Hour();
511             }else if(alarm_choose == 2){
512                 Set_Alarm_Minute();
513             }else if(alarm_choose == 3){
514                 Set_Alarm_Temp();
515             }else if(alarm_choose >= 4){
516                 updateActionTime();
517                 alarm_choose = 0;
518                 break;
519             }
520         }
521         delay(150);
522     }
523 }
524 
525 /** 获取 LM35 温度 */
526 void GetTemperatures(){
527     long a = analogRead(A3); // Get temperatures
528     Temperatures = 500.0*a/1023.0;
529     lcd.setCursor(9, 1) ;
530     lcd.print(Temperatures); //获取温度
531     lcd.setCursor(14, 1);
532     lcd.print((char)223); //显示 o 符号
```

```
533     lcd.setCursor(15, 1);
534     lcd.print("C"); //显示字母 C
535 }
536 
537 void setup(){
538     for(int i = 2;i <= 13; i++){
539         pinMode(i,OUTPUT);
540     }
541     digitalWrite(add, HIGH);
542     digitalWrite(minus, HIGH);
543     digitalWrite(choose, HIGH);
544     lcd.begin(16, 2); //初始化 LCD1602
545     //读取 DS1302 芯片的时间
546     rtc.writeProtect(false); // 关闭 DS1302 芯片写保护
547     rtc.halt(false);
548     //为 true 时 DS1302 暂停
549     Time t;
550     t=rtc.getTime();
551     set(t.year,t.mon,t.date,t.hour,t.min,t.sec); //设置 DS1302 芯片初始时间
552     // Time t;
553     // t.hour=10;t.min=47;t.sec=59;
554     // t.year=2025;t.mon=2;t.date=12;
555     // rtc.setTime(t.hour,t.min,t.sec); //设置 DS1302 芯片初始时间
556     // rtc.setDate(t.year,t.mon,t.date);
557     // set(t.year,t.mon,t.date,t.hour,t.min,t.sec);
558 }
559 
560 void loop() {
561     seconds = millis()/1000; //获取单片机当前运行时间
562 
563     // 检查按键活动
564     checkButtonActivity();
565 
566     // 检查是否需要熄屏
567     if (!screenOff && (millis() - lastActionTime > SCREEN_TIMEOUT)) {
568       turnOffScreen();
569     }
570 
571     Display(); //显示时间
572     Set_Clock(); //设置时间
573     Set_Alarm(); //设置闹钟
574     GetTemperatures(); //获取 LM35 温度
575     Point_Time_Alarm(); //正点蜂鸣
576     Clock_Alarm(); //闹钟时间蜂鸣
577     Temperatures_Alarm(); //超过指定温度报警
```

```
578
579     // 播放报警音乐（非阻塞）
580     playZheDaSong();
581
582     //将单片机的当前时间写到 DS1302 芯片中
583     rtc.setTime(hour,minute,second);
584     rtc.setDate(day,month,year);
585 }
```